

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу
«Операционные системы»

Группа: М8О-216БВ-24

Студент: Шитов А.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 10.12.25

Москва, 2025

Постановка задачи

Вариант 5.

Родительский процесс создаёт дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процессы должны быть представлены разными программами. Родительский процесс передаёт команды пользователя через `pipe1`, который связан со стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передаёт данные в родительский процесс через `pipe2`. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Пользователь вводит команды вида: «число<endline>». Далее это число передаётся от родительского процесса в дочерний. Дочерний процесс производит проверку на простоту. Если число составное, то это число записывается в файл. Если число отрицательное или простое, то дочерний и родительский процессы завершаются.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс.
- `int shm_open(const char *name, int oflag, mode_t mode)`; – создает или открывает объект разделяемой памяти.
- `int ftruncate(int fd, off_t length)`; – устанавливает размер объекта разделяемой памяти.
- `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset)`; – отображает разделяемую память в адресное пространство процесса.
- `sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value)`; – создает или открывает именованный семафор.
- `int sem_wait(sem_t *sem)`; – уменьшает значение семафора (операция P).
- `int sem_post(sem_t *sem)`; – увеличивает значение семафора (операция V).
- `int sem_close(sem_t *sem)`; – закрывает семафор.
- `int sem_unlink(const char *name)`; – удаляет именованный семафор из системы.
- `int munmap(void *addr, size_t length)`; – удаляет отображение разделяемой памяти.
- `int shm_unlink(const char *name)`; – удаляет объект разделяемой памяти из системы.
- `int execl(const char *path, const char *arg, ...)`; – заменяет образ текущей программы на указанную, принимая аргументы в качестве списка.
- `pid_t waitpid(pid_t pid, int *status, int options)`; – ожидает изменения состояния указанного дочернего процесса.

В рамках лабораторной работы создавалась программа, которая использует механизмы разделяемой памяти и семафоров для межпроцессного взаимодействия. Родительский процесс `parent.c` создаёт дочерний процесс (с помощью системного вызова `fork()`), который заменяет свой образ на программу `child.c` (с помощью системного вызова `execl()`).

Для организации взаимодействия между процессами используется разделяемая память (*shared memory*) и семафоры (*semaphores*). Родительский процесс создаёт область разделяемой памяти и семафор для синхронизации, получает у пользователя имя файла. Затем родительский процесс читает введённые числа и пишет их в разделяемую память. Дочерний процесс читает числа из разделяемой памяти и производит проверку на простоту. Отправляет код выполнения родительскому процессу через разделяемую память и в зависимости либо продолжает обработку с выводом числа в файл, либо оба процесса завершаются.

Код программы

main.c

```
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <stdio.h>

#define SHM_NAME "shared_memory"
#define SEM_DATA_RDY "sem_data"
#define SEM_RESPONSE_RDY "sem_response"
#define SHM_SIZE 1024

int main(int argc, char *argv[])
{
    if (argc != 1)
    {
        return 1;
    }

    char filename[256];
    const char msg[] = "Input the name of file: ";
    write(STDOUT_FILENO, msg, sizeof(msg));
    ssize_t bytes = read(STDIN_FILENO, filename, sizeof(filename) - 1);
    if (bytes <= 0)
    {
        const char error_msg[] = "Error: unable to get user data\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }
    filename[bytes - 1] = '\0';

    shm_unlink(SHM_NAME);
    sem_unlink(SEM_DATA_RDY);
    sem_unlink(SEM_RESPONSE_RDY);

    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1)
    {
        const char error_msg[] = "Error: unable to create shared memory\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }

    if (ftruncate(shm_fd, SHM_SIZE) == -1)
    {
        const char error_msg[] = "Error: unable to truncate shared memory\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }

    char *shm_ptr = mmap(0, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (shm_ptr == MAP_FAILED)
    {
        const char error_msg[] = "Error: unable to mmap shared memory\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }
}
```

```

}

sem_t *sem_data_rdy = sem_open(SEM_DATA_RDY, O_CREAT, 0666, 0);
sem_t *sem_response_rdy = sem_open(SEM_RESPONSE_RDY, O_CREAT, 0666, 0);

if (sem_data_rdy == SEM_FAILED || sem_response_rdy == SEM_FAILED)
{
    const char error_msg[] = "Error: unable to create semaphores\n";
    write(STDERR_FILENO, error_msg, sizeof(error_msg));
    exit(EXIT_FAILURE);
}

pid_t child_id = fork();
switch (child_id)
{
case -1:
{
    const char error_msg[] = "Error: unable to create child process\n";
    write(STDERR_FILENO, error_msg, sizeof(error_msg));
    exit(EXIT_FAILURE);
}
case 0:
{
    int exec_status = execl("./child", "child", NULL);
    if (exec_status == -1)
    {
        const char error_msg[] = "Error: unable to execl\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }
}
default:
{
    strncpy(shm_ptr, filename, SHM_SIZE - 1);
    shm_ptr[SHM_SIZE - 1] = '\0';
    sem_post(sem_data_rdy);

    const char msg[] = "Input numbers (q for quit):\n";
    write(STDOUT_FILENO, msg, sizeof(msg));

    char buffer[128];
    int bytes;
    while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) > 0)
    {
        buffer[bytes] = '\0';

        strncpy(shm_ptr, buffer, SHM_SIZE - 1);
        shm_ptr[SHM_SIZE - 1] = '\0';
        sem_post(sem_data_rdy);

        sem_wait(sem_response_rdy);

        char buffer2[128];
        strncpy(buffer2, shm_ptr, sizeof(buffer2) - 1);
        buffer2[sizeof(buffer2) - 1] = '\0';

        if (buffer2[0] == '1')
        {
            const char msg[] = "Done\n";
            write(STDERR_FILENO, msg, sizeof(msg));
            break;
        }
    }
}
}

```

```

    }

    int exec_child_status;
    wait(&exec_child_status);

    if (WIFEXITED(exec_child_status))
    {
        if (WEXITSTATUS(exec_child_status) != 0)
        {
            const char error_msg[] = "Error: child process terminated with error\n";
            write(STDERR_FILENO, error_msg, sizeof(error_msg));
            exit(EXIT_FAILURE);
        }
    }
    else
    {
        const char error_msg[] = "Error: the process is executing\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        exit(EXIT_FAILURE);
    }

    munmap(shm_ptr, SHM_SIZE);
    close(shm_fd);
    shm_unlink(SHM_NAME);
    sem_close(sem_data_rdy);
    sem_close(sem_response_rdy);
    sem_unlink(SEM_DATA_RDY);
    sem_unlink(SEM_RESPONSE_RDY);

    exit(EXIT_SUCCESS);
}
}

return 0;
}

```

child.c

```

#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>

#define SHM_NAME "shared_memory"
#define SEM_DATA_RDY "sem_data"
#define SEM_RESPONSE_RDY "sem_response"
#define SHM_SIZE 1024

typedef enum
{
    OK = 0,
    NOT_OK = 1
} status_code;

status_code check_positive_composite(const int number)
{
    if (number == 1)
    {

```

```

        return OK;
    }

    for (int i = 2; i * i <= number; ++i)
    {
        if (number % i == 0)
        {
            return OK;
        }
    }

    return NOT_OK;
}

int main(int argc, char *argv[])
{
    if (argc != 1)
    {
        return 1;
    }

    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    if (shm_fd == -1)
    {
        const char error_msg[] = "Error: unable to open shared memory\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        return 1;
    }

    char *shm_ptr = mmap(0, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (shm_ptr == MAP_FAILED)
    {
        const char error_msg[] = "Error: unable to mmap shared memory\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        return 1;
    }

    sem_t *sem_data_rdy = sem_open(SEM_DATA_RDY, 0);
    sem_t *sem_response_rdy = sem_open(SEM_RESPONSE_RDY, 0);

    if (sem_data_rdy == SEM_FAILED || sem_response_rdy == SEM_FAILED)
    {
        const char error_msg[] = "Error: unable to open semaphores\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        return 1;
    }

    sem_wait(sem_data_rdy);
    char filename[256];
    strncpy(filename, shm_ptr, sizeof(filename) - 1);
    filename[sizeof(filename) - 1] = '\0';

    int file = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (file == -1)
    {
        const char error_msg[] = "Error: unable to open file\n";
        write(STDERR_FILENO, error_msg, sizeof(error_msg));
        return 1;
    }

    char buffer[32];
    while (1)

```

```

{
    sem_wait(sem_data_rdy);
    strncpy(buffer, shm_ptr, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';

    if (buffer[0] == 'q')
    {
        strncpy(shm_ptr, "1", SHM_SIZE - 1);
        shm_ptr[SHM_SIZE - 1] = '\0';
        sem_post(sem_response_rdy);
        break;
    }

    char *endptr = NULL;
    int number = strtol(buffer, &endptr, 10);

    if (endptr == buffer || (*endptr != '\n' && *endptr != '\0'))
    {
        break;
    }

    status_code status = check_positive_composite(number);

    if (status == OK)
    {
        int i = 0;
        int num_copy = number;
        char number_buffer[32];

        do
        {
            number_buffer[i++] = num_copy % 10 + '0';
            num_copy /= 10;
        } while (num_copy > 0);

        int start = 0;
        int end = i - 1;

        while (start < end)
        {
            char temp = number_buffer[start];
            number_buffer[start] = number_buffer[end];
            number_buffer[end] = temp;
            ++start;
            --end;
        }

        number_buffer[i++] = '\n';
        write(file, number_buffer, i);

        strncpy(shm_ptr, "0", SHM_SIZE - 1);
        shm_ptr[SHM_SIZE - 1] = '\0';
        sem_post(sem_response_rdy);
    }
    else
    {
        strncpy(shm_ptr, "1", SHM_SIZE - 1);
        shm_ptr[SHM_SIZE - 1] = '\0';
        sem_post(sem_response_rdy);
        break;
    }
}

```

```
close(file);

munmap(shm_ptr, SHM_SIZE);
close(shm_fd);
sem_close(sem_data_rdy);
sem_close(sem_response_rdy);

return 0;
}
```


Протокол работы программы

```
test1.txt x
test1.txt
1 1
2 15
3 100
4

mrx@pc: ~/Desktop/OS/Lab3/src

mrx@pc:~/Desktop/OS/Lab3/src$ ./main
Input the name of file: /home/mrx/Desktop/OS/Lab3/src/test1.txt
Input numbers (q for quit):
1
15
100
-2
Done
```

```
test2.txt x
test2.txt
1 10
2 12
3 14
4 16
5 20
6 105
7 111

mrx@pc: ~/Desktop/OS/Lab3/src

mrx@pc:~/Desktop/OS/Lab3/src$ ./main
Input the name of file: /home/mrx/Desktop/OS/Lab3/src/test2.txt
Input numbers (q for quit):
10
12
14
16
20
105
111
13
Done
```

Strace:

```
strace -f ./main
execve("./main", ["/main"], 0x7ffc2128e2d8 /* 78 vars */) = 0
brk(NULL) = 0x641c4e878000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7934a2d0d000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=79219, ...}) = 0
mmap(NULL, 79219, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7934a2cf9000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7934a2a00000
mmap(0x7934a2a28000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7934a2a28000
mmap(0x7934a2bb0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7934a2bb0000
mmap(0x7934a2bff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7934a2bff000
mmap(0x7934a2c05000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7934a2c05000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7934a2cf6000
arch_prctl(ARCH_SET_FS, 0x7934a2cf6740) = 0
set_tid_address(0x7934a2cf6a10) = 182173
set_robust_list(0x7934a2cf6a20, 24) = 0
rseq(0x7934a2cf7060, 0x20, 0, 0x53053053) = 0
mprotect(0x7934a2bff000, 16384, PROT_READ) = 0
mprotect(0x641c243c8000, 4096, PROT_READ) = 0
mprotect(0x7934a2d4b000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7934a2cf9000, 79219) = 0
write(1, "Input the name of file: \0", 25Input the name of file: ) = 25
read(0, /home/mrx/Desktop/OS/Lab3/src/test1.txt
/home/mrx/Desktop/OS/Lab3/src/te"..., 255) = 40
unlink("/dev/shm/shared_memory") = -1 ENOENT (No such file or directory)
unlink("/dev/shm/sem.sem_data") = -1 ENOENT (No such file or directory)
unlink("/dev/shm/sem.sem_response") = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/dev/shm/shared_memory", O_RDWR|O_CREAT|O_NOFOLLOW|O_CLOEXEC, 0666) = 3
ftruncate(3, 1024) = 0
mmap(NULL, 1024, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7934a2d0c000
openat(AT_FDCWD, "/dev/shm/sem.sem_data", O_RDWR|O_NOFOLLOW|O_CLOEXEC) = -1 ENOENT (No such file or directory)
getrandom("\xd6\x93\xff\x23\x03\xf8\x5e\x87", 8, GRND_NONBLOCK) = 8
newfstatat(AT_FDCWD, "/dev/shm/sem.w8kUQR", 0x7ffee86ef860, AT_SYMLINK_NOFOLLOW) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/dev/shm/sem.w8kUQR", O_RDWR|O_CREAT|O_EXCL|O_NOFOLLOW|O_CLOEXEC, 0666) = 4
write(4, "\0\0\0\0\0\0\0\0\200\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0", 32) = 32
mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7934a2d0b000
link("/dev/shm/sem.w8kUQR", "/dev/shm/sem.sem_data") = 0
fstat(4, {st_mode=S_IFREG|0664, st_size=32, ...}) = 0
getrandom("\x2f\xeb\x62\x29\x92\x8b\xfd\x25", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x641c4e878000
brk(0x641c4e899000) = 0x641c4e899000
unlink("/dev/shm/sem.w8kUQR") = 0
close(4) = 0
```

[illegible]

```
[pid 182186] brk(0x5678c2e41000) = 0x5678c2e41000
[pid 182186] mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7b60ea8bf000
[pid 182186] close(4) = 0
[pid 182186] openat(AT_FDCWD, "/dev/shm/sem.sem_response", O_RDWR|O_NOFOLLOW|O_CLOEXEC) = 4
[pid 182186] fstat(4, {st_mode=S_IFREG|0664, st_size=32, ...}) = 0
[pid 182186] mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7b60ea8be000
[pid 182186] close(4) = 0
[pid 182186] openat(AT_FDCWD, "/home/mrx/Desktop/OS/Lab3/src/test1.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
```

```
[pid 182186] futex(0x7b60ea8bf000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY15
```

```
<unfinished ...>
```

```
[pid 182173] <... read resumed>"15\n", 127) = 3
```

```
[pid 182173] futex(0x7934a2d0b000, FUTEX_WAKE, 1) = 1
```

```
[pid 182173] futex(0x7934a2d0a000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>
```

```
[pid 182186] <... futex resumed>) = 0
```

```
[pid 182186] write(4, "15\n", 3) = 3
```

```
[pid 182186] futex(0x7b60ea8be000, FUTEX_WAKE, 1 <unfinished ...>
```

```
[pid 182173] <... futex resumed>) = 0
```

```
[pid 182186] <... futex resumed>) = 1
```

```
[pid 182173] read(0, <unfinished ...>
```

```
[pid 182186] futex(0x7b60ea8bf000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY7
```

```
<unfinished ...>
```

```
[pid 182173] <... read resumed>"7\n", 127) = 2
```

```
[pid 182173] futex(0x7934a2d0b000, FUTEX_WAKE, 1) = 1
```

```
[pid 182173] futex(0x7934a2d0a000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>
```

```
[pid 182186] <... futex resumed>) = 0
```

```
[pid 182186] futex(0x7b60ea8be000, FUTEX_WAKE, 1 <unfinished ...>
```

```
[pid 182173] <... futex resumed>) = 0
```

```
[pid 182186] <... futex resumed>) = 1
```

```
[pid 182173] write(2, "Done\n\0", 6 <unfinished ...>
```

```
Done
```

```
[pid 182186] close(4 <unfinished ...>
```

```
[pid 182173] <... write resumed>) = 6
```

```
[pid 182173] wait4(-1, <unfinished ...>
```

```
[pid 182186] <... close resumed>) = 0
```

```
[pid 182186] munmap(0x7b60ea8c0000, 1024) = 0
```

```
[pid 182186] close(3) = 0
```

```
[pid 182186] munmap(0x7b60ea8bf000, 32) = 0
```

```
[pid 182186] munmap(0x7b60ea8be000, 32) = 0
```

```
[pid 182186] exit_group(0) = ?
```

```
[pid 182186] +++ exited with 0 +++
```

```
<... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 182186
```

```
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=182186, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
```

```
munmap(0x7934a2d0c000, 1024) = 0
```

```
close(3) = 0
```

```
unlink("/dev/shm/shared_memory") = 0
```

```
munmap(0x7934a2d0b000, 32) = 0
```

```
munmap(0x7934a2d0a000, 32) = 0
```

```
unlink("/dev/shm/sem.sem_data") = 0
```

```
unlink("/dev/shm/sem.sem_response") = 0
```

```
exit_group(0) = ?
```

```
+++ exited with 0 +++
```

Вывод

В ходе выполнения лабораторной работы была успешно реализована программа межпроцессного взаимодействия с использованием механизмов разделяемой памяти и семафоров.